RANK-S

Banking Transaction System Proposal

Prepared By:
Rank-S Team

THE OVERVIEW

The Banking Transaction System is a Java-based application designed to manage bank accounts and perform basic financial operations, including account creation, balance inquiries, deposits, withdrawals, transfers, and transaction history. The project applies Object-Oriented Programming (OOP), collections, exception handling, and persistent data storage.
Software testing will be integrated throughout development to verify business rules, identify defects early, and help ensure transaction correctness and data consistency. The system is intended for educational use only and will not process real money or connect to real banking networks.

PROBLEM STATEMENT

When we think about banking applications, we see the need for account management and reliable transaction processing. Account management and transaction processing keep data consistent. Stop wrong financial moves. If deposits, withdrawals, or transfers go wrong, account balances can become incorrect. Data can become messy.
This project is about building a Java-based Banking Transaction System that lets people do account management, check balances, make deposits, take out money, move money, and see transaction history. The project also cares about software quality. We will test business rules, how the system handles errors, how transaction processing stays safe, and how data is stored. Both manual and automated tests will help find problems and make sure things work as they should. The project does not do banking operations or real money moves.

PROJECT OBJECTIVES

S1.Account management features must be developed. I think these features should include deposits, withdrawals, transfers, and balance inquiries.

S2.Apply programming principles. Use collections to store data. Use exception handling to manage errors. Use file or database handling for data storage.

S3.Assign unique account numbers to every account. Keep a transaction history.

S4.Validate transaction amounts and account details. Prevent invalid operations. Prevent. Transfers that exceed account balance.

S5.Verify transaction integrity. Verify data consistency. Verify persistence. Use testing to confirm all of these.

S6.Use Jira to track tasks, testing activities, defects, and results.

PROJECT SCOPE

Development In Scope

Create accounts and store or retrieve account information.

Display account. Allow deposits, withdrawals, and transfers between accounts.

Record the type of transaction, the amount, the date, and the accounts involved.

Reject amounts and stop operations when there are not enough funds, and check that the destination account exists.

Save account and transaction data so the information stays available even after the application is restarted.

Testing In Scope

Review requirements, design, and source code

Test the application manually for normal, negative, and boundary cases.

Run unit tests for business logic and integration tests for components.

Perform regression testing, retest defects, and verify that data is saved correctly.

Use Selenium WebDriver to test UI workflows if a web interface is used.

Use Postman to test backend operations if a REST API is implemented.

Track testing activities, defects, priorities, and results using Jira.

Out of Scope

Connect to banks, payment networks or handle actual financial transactions.

Issue bank. Support loans, investments or other advanced banking services.

Work with money or actual financial data.

Run performance, penetration or security testing unless specifically approved.

.

PROPOSED TECHNOLOGY STACK

The following choices are. Have not yet been confirmed. The team will finalize the architecture and storage approach after the team considers assignment requirements, team skills and available time.

- Programming Language: Java (Required)

- Architecture: Console application or web UI, with REST API (Not Decided)

- Data Storage: File storage, SQLite or MySQL (Not decided)

- Unit Testing: JUnit (Proposed)

- UI Automation: Selenium WebDriver (If a web UI is selected)

- API Testing: Postman (If a REST API's implemented)

- Test and Task Management: Jira (Required by the project plan)

If Selenium UI testing is mandatory the team should pick a web interface. A REST API is recommended if the team wants to demonstrate Postman API testing.

SOFTWARE TESTING STRATEGY

We will make sure that Testing follows a risk-based approach focusing on transaction correctness and data integrity.

We will make sure that Static Testing reviews requirements, acceptance criteria, design and source code before any execution.

We will make sure that Manual Testing runs negative, validation and boundary test cases.

We will make sure that Unit Testing verifies methods and business rules using JUnit.

We will make sure that Integration Testing verifies interactions between accounts, transaction operations, and storage.

We will make sure that UI Automation tests support user workflows with Selenium WebDriver if it applies.

We will make sure that API Testing verifies endpoint behavior, response codes, response data, and validation errors with Postman if it applies.

We will make sure that Regression Testing reruns tests after code changes and defect fixes.

We will make sure that Test Management records test outcomes, defects, evidence, and status in Jira.

We will make sure that key checks include ensuring rejected transactions do not incorrectly change balances, successful transfers update both accounts, transaction history matches completed operations, and saved data remains correct after restarting the application.

EXPECTED DELIVERABLES

Development Deliverables

A Java application that includes account and transaction classes.

Handling of exceptions for transactions.

Processing of files or databases for account and transaction data.

Account management features, including balance inquiry, deposit, withdrawal, and transfer functions.

Transaction and Input validation.

Testing Deliverables

Test plan, test scenarios, test cases, and test data.

Findings from a review and documented defects.

Results from testing, including positive, negative, and boundary scenarios.

Results from unit and integration tests.

Selenium tests. The results from running them, if they are used.

Postman. The results from running them, if they are used.

Jira records for tasks, defects, priorities, and status.

Results from regression testing and evidence of defect retesting.

Final test summary. Requirements traceability matrix.